

Agentic AI System for Enterprise Cloud Migration

Timothy Wong • Alessandro Abati

London Stock Exchange Group (LSEG)

7 Oct 2026

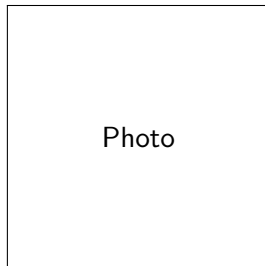


About the Speakers



Timothy Wong

Tim is an AI/ML engineer at LSEG with over ten years experience in enterprise data science applications. He hosted talks at EARL 2016, 17, 19 & 24.



Alessandro Abati

placeholder



LSEG

LSEG is a leading financial markets infrastructure and data provider operating connected businesses to serve customers across the entire financial markets value chain.

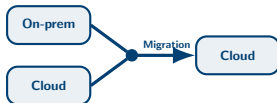
- ▶ **London Stock Exchange**
- ▶ **FTSE Russell:** Indices across asset classes, including FTSE 100, FTSE 250, Russell 2000, and ESG indices.
- ▶ **Data & Analytics:** Workspace platform, pricing, reference data.
- ▶ **Clearing:** LCH, multi-asset class clearing house.
- ▶ **Scale:** ~26 000 employees across many countries.

1. **Business Requirements and Context**
2. **Agentic Technical Deep Dive**
 - 2.1 **Data Preprocessing**
 - 2.2 **Inside Agentic Retrieval**
 - 2.3 **Agentic Orchestration**
 - 2.4 **Agents' Memory**
 - 2.5 **Agents' Tools**
 - 2.6 **E2E Orchestration Phases**
3. **Application Overview**

Business Requirements and Context

Co-Migrate supports the applications' journey to the cloud, turning a slow, manual migration effort into an AI-assisted one.

Migration patterns



From codebase to discovery report

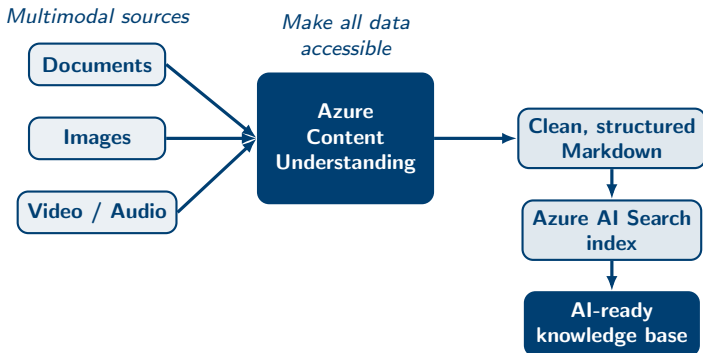


- **Discovery first:** every migration starts with a discovery stage to map dependencies and spot risks.

The problems Co-Migrate solves

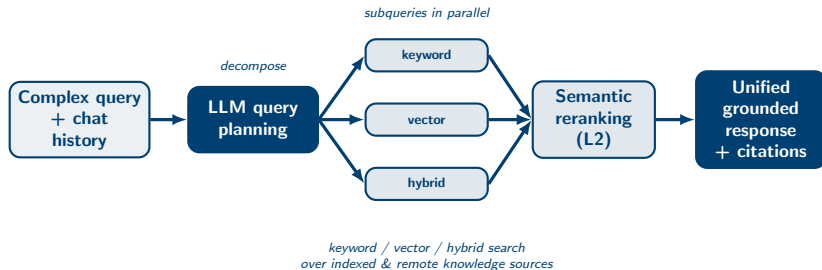
- **The bottleneck:** teams can spend up to **53 weeks** on discovery alone - slow, manual and costly.
- **Risk matters early:** spotting risks and complexity up front is key to a successful migration.

Data Preprocessing - Making Documentation AI-Ready



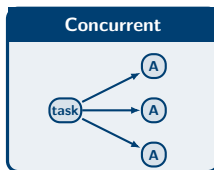
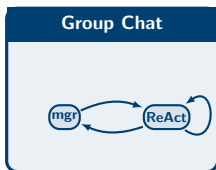
- **Content Understanding:** one analyzer for *any* modality - OCR, layout, tables and figure descriptions turn messy files into clean Markdown.
- **Make all data accessible:** charts, diagrams and scans become searchable text, so nothing is lost to downstream agents.
- **Azure AI Search:** content is chunked and indexed into an AI-ready knowledge base for agentic retrieval.

Inside Agentic Retrieval - A Multi-Query Pipeline



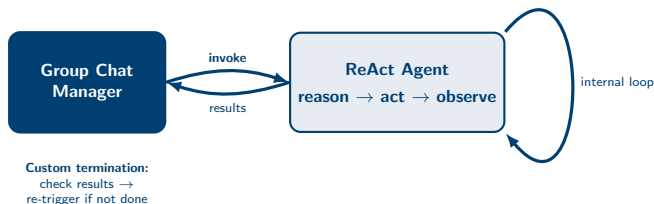
- ▶ **Beyond classic RAG:** handles multi-part questions, chat context, typos and paraphrasing that a single query can't.
- ▶ **Query planning:** an LLM breaks the query down into focused subqueries that run *in parallel*, each semantically reranked.
- ▶ **Grounded output:** best results are merged into one response with source citations and an activity log for agents.

Agentic Orchestration - Overview



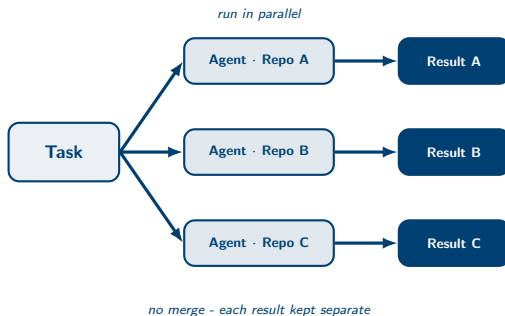
- ▶ **Framework:** Microsoft **Semantic Kernel** agent orchestration, tailored to our use case.
- ▶ **Two patterns:** *Group Chat* (1 manager + 1 ReAct agent, custom termination) and *Concurrent* (parallel, per-repo agents).
- ▶ **Writer/Reviewer** is a *Round-Robin* flavour of Group Chat - same orchestration, alternating turns.

Agentic Orchestration - Group Chat



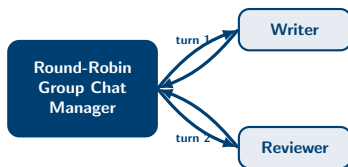
- **Our setup:** *one* manager coordinating a *single* ReAct agent - not a multi-agent debate.
- **Why a manager:** a ReAct agent's reason-act loop is trained to stop early; the manager inspects its results and *re-triggers* the loop when the task isn't complete.
- **Net effect:** a custom termination strategy on top of the agent's own reason-act loop.

Agentic Orchestration - Concurrent



- ▶ **Our setup:** the task fans out to one agent *per repo*, all running in parallel.
- ▶ **No aggregation:** results are *not* combined - each agent only needs its own repo's context.
- ▶ **Benefits:** parallelism for speed, plus clean isolation - no cross-repo context bleed.

Agentic Orchestration - Writer/Reviewer



What Round-Robin overrides

`SelectNextAgent` - picks the next agent by a fixed **cyclic order** (a round-robin cursor).

Everything else is inherited from the base `GroupChatManager`.

- ▶ **Same pattern:** reuses the *Group Chat* orchestration with the `RoundRobinGroupChatManager` flavour - no custom manager logic needed.
- ▶ **Deterministic turns:** Writer drafts, Reviewer critiques, alternating for a fixed number of rounds (writer gets the last word).
- ▶ **Benefits:** a built-in quality gate - iterative critique lifts output quality with predictable turns and no custom manager logic.

- ▶ **What it is:** *placeholder* -
- ▶ **How it works:** *placeholder* - co-located `.agent.md` prompt, agent-owned memory, shared memory.
- ▶ **Where used:** *placeholder* - discovery

- ▶ **Agent anatomy:** factory `get_X_agent()`, co-located `.agent.md` prompt, agent-owned tools.
- ▶ **Tool catalogue:** `discovery_memory`, `file_system`, `search`, `grounding`, `ado_docs`, `sdp_docs`.
- ▶ **Shared middleware:** *placeholder* - Agent / Function / Chat / TokenUsage logging.

Agent-as-a-tool Pattern

- ▶ **Concept:** *placeholder* - exposing an agent as a callable tool for another agent.
- ▶ **Why:** *placeholder* - composition, encapsulation, reuse.
- ▶ **Example:** *placeholder*.

E2E Orchestration Phases - Overview

Discovery → Deep Analysis → Enterprise Context → UAQ →
Reporting → Cleanup

placeholder - how the phases hand off state end-to-end.

- ▶ **Goal:** *placeholder*.
- ▶ **Agents & tools:** *placeholder*.
- ▶ **Output:** *placeholder* - discovery memory.

Deep Analysis Phase

- ▶ **Goal:** *placeholder.*
- ▶ **Orchestration:** concurrent batches.
- ▶ **Output:** *placeholder.*

Enterprise Context Phase

- ▶ **Goal:** *placeholder* - enrich with LSEG/enterprise context.
- ▶ **Sources:** *placeholder*.
- ▶ **Output:** *placeholder*.

Unified Assessment Questionnaire (UAQ) Phase

- ▶ **Goal:** *placeholder* - Unified Assessment Questionnaire.
- ▶ **Orchestration:** category-scoped concurrent agents.
- ▶ **Output:** *placeholder*.

Reporting Phase

- ▶ **Goal:** *placeholder* - final recommendation report.
- ▶ **Orchestration:** writer/reviewer loop.
- ▶ **Output:** *placeholder*.

- ▶ **High-level architecture:** *placeholder* - diagram.
- ▶ **Stack:** Python 3.12, FastAPI, Poetry; React 19 frontend.
- ▶ **Data flow:** ingestion → preprocessing → agentic retrieval → orchestration.

- ▶ **Azure services:** *placeholder* - Blob / Table / Queue, AI Search, Key Vault, Foundry OpenAI.
- ▶ **Deployment:** *placeholder*.
- ▶ **Air-gapped considerations:** offline Mermaid CLI rendering, no remote fallback.

- ▶ **API:** *placeholder* - FastAPI endpoints.
- ▶ **Reliability:** `retry.py` `ErrorClass` (transient / deterministic / partial), exponential backoff + jitter.
- ▶ **Observability:** *placeholder* - OpenTelemetry token metrics.

- ▶ **Stack:** React 19.
- ▶ **Key views:** *placeholder.*
- ▶ **UX flow:** *placeholder.*

Thank you - Questions?